Sherlock: "EasyMoney" - Investigation Writeup

Background

John is an employee at a mid-sized tech company, working as a Senior IT Support Specialist. His passion for finding extra income led him to click on an enticing giveaway link online. When the giveaway yielded nothing, he grew suspicious that something had gone wrong with his machine. Concerned about potential compromise, John reached out to request a formal investigation.

Scope

The investigation was conducted on a forensic copy of the C:\ drive from the compromised system, acquired using KAPE. Fifteen guiding tasks were provided to structure the investigation.

Tools Used

Eric Zimmerman Tools

- EvtxECmd.exe — Event log (.evtx) parsing
- PECmd.exe — Windows Prefetch (.pf) file parsing
- MFTECmd.exe — Master File Table ($MFT) and USN Journal ($J) parsing
- AmcacheParser.exe — Amcache hive parsing
- SrumECmd.exe — SRUM (System Resource Usage Monitor) database parsing
- TimelineExplorer — CSV output viewer and sorter
- Registry Explorer — Registry hive browser

Other Tools

- DB Browser (SQLite) — Browser history database analysis
- Ghidra — Static malware analysis and reverse engineering

Building the Timeline

At the start of the investigation, a timeline of the incident was built using the background story and guiding questions. The following commands were used to collect artifact data from the

system:

```
PECmd.exe -d "C:
\Users\Investigate\Desktop\sherlock\EasyMoney\C\Windows\Prefetch" --csv C:
\Users\Investigate\Desktop\sherlock\EasyMoney
EvtxECmd.exe -d "C:
\Users\Investigate\Desktop\sherlock\EasyMoney\C\Windows\System32\winevt\logs"
--csv C:\Users\Investigate\Desktop\sherlock\EasyMoney
MFTECmd.exe -f "C:\Users\Investigate\Desktop\sherlock\EasyMoney\C\$Extend\$J"
--csv C:\Users\Investigate\Desktop\sherlock\EasyMoney
MFTECmd.exe -f "C:\Users\Investigate\Desktop\sherlock\EasyMoney\C\$MFT" --csv
C:\Users\Investigate\Desktop\sherlock\EasyMoney
```

The following registry hives were also loaded into Registry Explorer:

```
C:\Windows\System32\config\DEFAULT
C:\Windows\System32\config\SYSTEM
C:\Windows\System32\config\SOFTWARE
C:\Windows\System32\config\SECURITY
C:\Windows\System32\config\SAM
C:\Users\Administrator\NTUSER.DAT
```

Investigation Tasks

Task 1 - Time of Malicious Shortcut Execution

Based on the background story, a giveaway-themed file was downloaded and executed by the user. To find the exact execution time, Registry Explorer was used to examine the NTUSER.DAT hive.

Navigating to:

```
ROOT\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer\UserAssist
```

ROOT	0	10	2025-01-25 20:20:27
SOFTWARE	0	12	2025-01-26 10:25:31
Microsoft	0	65	2025-01-26 18:09:59
Windows	0	8	2025-01-26 10:03:52
CurrentVersion	0	66	2025-01-26 16:10:00
Explorer	12	46	2025-01-26 16:39:21
UserAssist	0	9	2025-01-23 22:50:01

The UserAssist key revealed that a .lnk file named 2025-GiveAways.lnk was executed by the user:

Program Name	Last Executed	Run Counter
	=	=
{User Pinned}\TaskBar\File Explorer.lnk	2025-01-26 18:09:36	22
Microsoft.Windows.Explorer	2025-01-26 18:09:36	22
C:\Users\Administrator\Desktop\Yandex.lnk	2025-01-26 16:59:26	1
Yandex.OW7P5ENQTG2X3IMNGUOA2AKRNE	2025-01-26 16:59:26	1
C:\Users\Administrator\Desktop\Microsoft Edge.lnk	2025-01-26 16:53:52	3
MSEdge	2025-01-26 16:53:52	3
{ProgramFilesX86}\FOXIT SOFTWARE\FOXIT PDF READER\FOXITPDFREADER.EXE	2025-01-26 16:39:44	1
C:\Users\Administrator\Downloads\2025-GiveAways.lnk	2025-01-26 16:17:15	1

`C:\Users\Administrator\Downloads\2025-GiveAways.lnk`

Answer: `2025-01-26 16:17:15 (UTC)`

Task 2 - Full Path of the Initial Payload

Using the execution timestamp found in Task 1, the $MFT and $J (USN Journal) artifacts were examined for any suspicious file activity in the same timeframe. A file named svch0st.exe was found to have been created shortly after the .lnk execution. The name is a deliberate imitation of the legitimate Windows process svchost.exe — a common attacker technique to avoid suspicion.

.\Temp	svch0st.exe	.exe				…	2025-01-26 16:17:17		2025-01-26 16:17:54

The $J file confirmed the file's creation and provided its full path:

Answer: `C:\Temp\svch0st.exe`

Task 3 - Timestamp of Shell Access

The question hints that the payload established a shell connection to the attacker upon execution. The Last Modified timestamp in the $MFT output for svch0st.exe indicates when it was first executed and a connection was established.

Parent P…	File Name	Ext…	…	…	…	…	Created0x10	…	Last Modified0x10
	svch0st						=		=
.\Temp	svch0st.exe	.exe				…	2025-01-26 16:17:17		2025-01-26 16:17:54

Answer: `2025-01-26 16:17:54 (UTC)`

Task 4 - Attacker's Package Enumeration Command

To identify the enumeration command, the parsed event logs (EVTX) were searched for PowerShell activity. The command Get-Package is a well-known attacker technique for enumerating installed software on a Windows system. Filtering the CSV output for this command confirmed its use:

Get-Package × Find

Payload Data1
HostApplication=C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -Command Get-Package

Answer: `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -Command Get-Package`

Task 5 - Vulnerable Application Identified by Attacker

To answer this question, it was necessary to look ahead in the incident timeline to understand the attacker's objective. Reviewing the event logs for credential-access activity revealed multiple Windows Vault read attempts. Most returned zero credentials, but a small number successfully returned one credential. Examining the first successful read, the responsible executable was identified as part of the Yandex Browser application, and the targeted vault was labeled "Yandex-Browser".

CountOfCredentialsReturned: 0	…	MicrosoftAccount:user=02pbbqlfqsddbxdh
CountOfCredentialsReturned: 0	…	MicrosoftAccount:user=02pbbqlfqsddbxdh
CountOfCredentialsReturned: 0	…	MicrosoftAccount:user=02pbbqlfqsddbxdh
CountOfCredentialsReturned: 0	…	MicrosoftAccount:user=02pbbqlfqsddbxdh
CountOfCredentialsReturned: 0	…	MicrosoftAccount:user=02qkqebvltntyano
CountOfCredentialsReturned: 0	…	MicrosoftAccount:user=02qkqebvltntyano
CountOfCredentialsReturned: 0	…	MicrosoftAccount:user=02qkqebvltntyano
CountOfCredentialsReturned: 0	…	MicrosoftAccount:user=02qkqebvltntyano
CountOfCredentialsReturned: 1	…	Yandex.Browser
CountOfCredentialsReturned: 0	…	WindowsLive:(token):name=02nqqyyjfqqpokcg;serviceuri=
CountOfCredentialsReturned: 0	…	WindowsLive:(cert):name=02nqqyyjfqqpokcg;serviceuri=*
CountOfCredentialsReturned: 1	…	WindowsLive:target=virtualapp/didlogical
CountOfCredentialsReturned: 0	…	WindowsLive:(token):name=02qkqebvltntyano;serviceuri=
CountOfCredentialsReturned: 0	…	WindowsLive:(cert):name=02qkqebvltntyano;serviceuri=*
CountOfCredentialsReturned: 1	…	WindowsLive:target=virtualapp/didlogical

Answer: `YandexBrowser`

Task 6 - Vulnerable Application Version

To determine the exact version of the installed browser, Registry Explorer was used to examine the NTUSER.DAT hive at the following path:

```
NTUSER.DAT\Software\Yandex\YandexBrowser
```

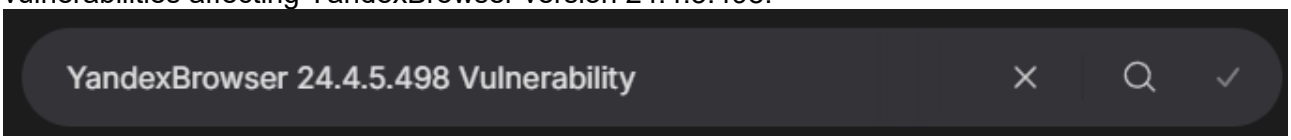
This key contains the YPV (Yandex Package Version) value, which provides the installed version:

Value Name	Value Type	Data
name	RegSz	Yandex
country_code	RegSz	US
yasend_install	RegSz	true
lang	RegSz	en
UninstallString	RegSz	C:\Users\Administrator\AppData\Local\Yandex\YandexBrowser\Application\...
pv	RegSz	55.0.2883.95
today_startup_count	RegDword	3
metricsid_launches_after_update	RegSz	3
ypv	RegSz	24.4.5.498
clid	RegSz	2372568
metricsid	RegSz	18165bd3-d915-4d2e-906b-d7765487c7c0
metricsid_updatedate	RegSz	1737887155
metricsid_enabledate	RegSz	1737887155
metricsid_installdate	RegSz	1737887155
last_shutdown_time	RegQword	13382384381307251
last_startup_time	RegQword	13382384368130095
firstinstalltime	RegSz	13382360754724089
installtime	RegSz	13382360754724089
InstallerProgress	RegDword	100
browser.background_update.allowed	RegDword	1
UsageStatsInSample	RegDword	1
usagestats	RegDword	1
InstallerError	RegDword	0
InstallerResult	RegDword	0
InstallerSuccessLaunchCmdLine	RegSz	"C:\Users\Administrator\AppData\Local\Yandex\YandexBrowser\Application\...
UninstallArguments	RegSz	--uninstall --verbose-logging
ap	RegSz	

Answer: `24.4.5.498`

Task 7 - CVE Associated with the Vulnerability

With the application name and version confirmed, a search was conducted for known vulnerabilities affecting YandexBrowser version 24.4.5.498.



The National Vulnerability Database (NVD) entry at https://nvd.nist.gov/vuln/detail/CVE-2024-6473 describes a DLL Hijacking vulnerability affecting all versions of Yandex Browser for Desktop prior to 24.7.1.380.
The vulnerability is caused by the use of an untrusted search path, allowing an attacker to place a malicious DLL in a directory that the browser loads from.
What is untrusted search path? https://cwe.mitre.org/data/definitions/426.html explain it good.

Description

The product searches for critical resources using an externally-supplied search path that can point to resources that are not under the product's direct control.

Extended Description

This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.
Some of the most common variants of untrusted search path are:

Answer: `CVE-2024-6473`

Task 8 - Legitimate Binary Used for Payload Delivery

certutil.exe is a well-known Living-off-the-Land binary (LOLBin) that is commonly abused by attackers to download files while evading detection.
This was confirmed by examining the CryptnetUrlCache - a folder where certutil.exe caches files it downloads, along with their source URLs. The following directories were inspected:

```
C:\Users\Administrator\AppData\LocalLow\Microsoft\CryptnetUrlCache\MetaData
C:\Users\Administrator\AppData\LocalLow\Microsoft\CryptnetUrlCache\Content
```

The following PowerShell command was used to extract the download URLs from the metadata files:

```
Get-ChildItem | ForEach-Object {
  $content = Get-Content $_.FullName -Encoding Unicode
  if ($content -match "http") {
    Write-Host "File: $($_.Name)" -ForegroundColor Cyan
    $content | Select-String -Pattern "http[s]?://[^\s]+" -AllMatches | %
{ $_.Matches.Value }
  }
}
```

```
File: 94C18AA77707A64853AFFBE6D6382F75
http://crl.starfieldtech.com/sfroot-g2.crl"259-618e505e9b66c"
File: 9CB4373A4252DE8D2212929836304EC5_1AB74AA2E3A56E1B8AD8D3FEC287554E
http://ocsp.globalsign.com/codesigningrootr45/MFEwTzBNMEswSTAJBgUrDgMCGgU
EHe9DgW3WOu2HUdhUx4%2Fde0%3D"70e1d52117fe97c359abcd5b3f3005619e2cdaa0"
File: A16B2E6DE64B13EDF2C00F32C4559930
http://18.192.12.126:8000/wldp.dll
File: B039FEA45CB4CC4BBACFC013C7C55604_50D7940D5D3FEDD8634D83074C7A46A3
http://ocsp2.globalsign.com/rootr3/ME4wTDBKMEgwRjAJBgUrDgMCGgUABBT1nGh%2F
GI71DM6hVc%3D"528cef1059387763bc41c3970e1228e04b0067a7"
File: B398B80134F72209547439DB21AB308D_D14B79B440CDC26D7D21C81855E2C04D
http://ocsp.digicert.com/MFEwTzBNMEswSTAJBgUrDgMCGgUABBSAUQYBMq2awn1Rh6Dc
7E%3D
File: C42D54F81A7A41980830652790DA75D8
http://ocsp.digicert.com/MFEwTzBNMEswSTAJBgUrDgMCGgUABBSXi0cW5bD2WLrmnasV
%3D
File: C8E534EE129F27D55460CE17FD628216_1130D9B25898B0DB0D4F04DC5B93F141
http://ocsp.digicert.com/MFEwTzBNMEswSTAJBgUrDgMCGgUABBTfIs%2BLjDtGwQ09XE
b2Trtk%3D
File: D7833C286363AD25C70511661A83D581_87080482E227A81FEA3DE5A553C20422
http://ocsp.sectigo.com/MFIwUDBOMEwwSjAJBgUrDgMCGgUABBQ5suEceKjAJbxseAmHF
File: DE69F438F13416BEDB3F9D0DBC8165A8
http://18.192.12.126:8000/yanda.tmp
```

The output revealed two files downloaded from the IP address 18.192.12.126. A reputation check on AbuseIPDB confirmed the IP is associated with an Amazon Web Services (AWS) hosted server being used as a C2 server.

18.192.12.126 was not found in our database

ISP	A100 ROW GmbH
Usage Type	Data Center/Web Hosting/Transit
ASN	Unknown
Hostname(s)	ec2-18-192-12-126.eu-central-1.compute.amazonaws.com
Domain Name	amazon.com
Country	🇩🇪 Germany
City	Frankfurt am Main, Hesse

More to that, based on the timeline of the $J file, we see the use of Certutil.exe with the creatin of the suspected payloads:

2025-01-26 16:37:00		yanda.tmp	.tmp	118348
2025-01-26 16:37:00		yanda.tmp	.tmp	118348
2025-01-26 16:37:00		CERTUTIL.EXE-28F1E0C1.pf	.pf	100807
2025-01-26 16:37:00		CERTUTIL.EXE-28F1E0C1.pf	.pf	100807

Answer: `certutil.exe`

Task 9 - Malicious PE File

The previous task identified two files downloaded via certutil.exe. Given that CVE-2024-6473 is a DLL Hijacking vulnerability, the malicious payload is expected to be a DLL. Cross-referencing the downloaded files with the $MFT output confirms that wldp.dll was placed in the YandexBrowser application directory - the exact location the browser searches for DLLs at launch.

Parent Path	File Name
Yandex	
.\Users\Administrator\AppData\Local\Yandex\YandexBrowser\Application	wldp.dll

Answer: `wldp.dll`

Task 10 - SHA-256 Hash of the Malicious File

The malicious wldp.dll was cached in the CryptnetUrlCache Content folder under the filename A16B2E6DE64B13EDF2C00F32C4559930. The SHA-256 hash was obtained using the following PowerShell command:

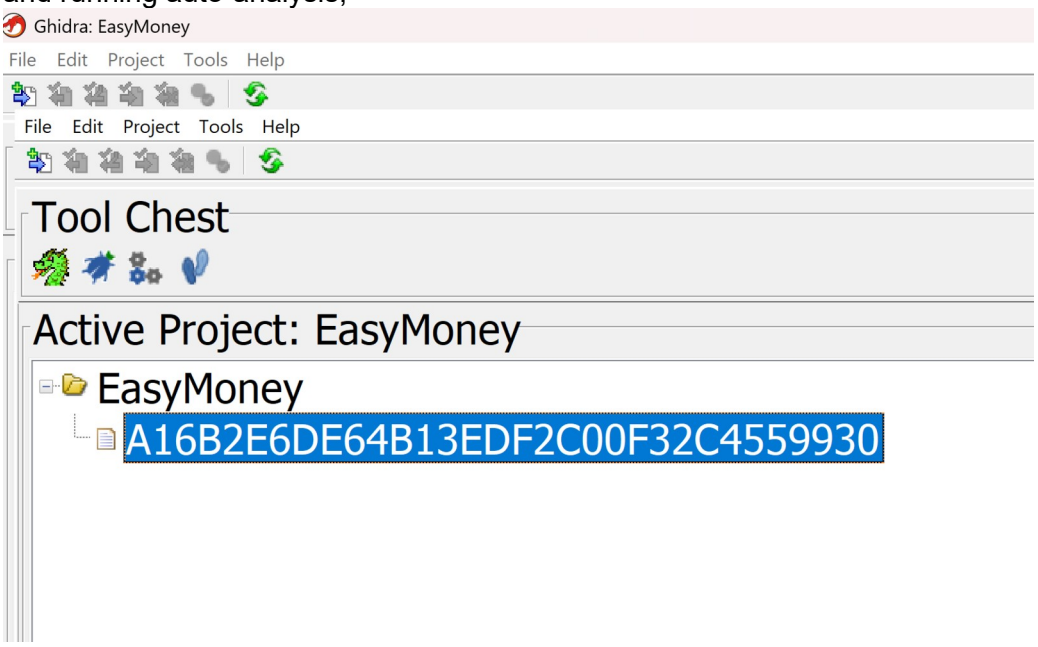
```
Get-FileHash "..\Content\A16B2E6DE64B13EDF2C00F32C4559930"

Algorithm       Hash
---------       ----
SHA256          A1A17EBD90610D808E761811D17DA3143F3DE0D4CC5EE92BD66000DCA87D9270
```

```
Get-FileHash "..\Content\A16B2E6DE64B13EDF2C00F32C4559930"
```

Answer:
A1A17EBD90610D808E761811D17DA3143F3DE0D4CC5EE92BD66000DCA87D9270

Task 11 - Cumulative Sleep Delays (Milliseconds)

To answer this question, Ghidra was used to statically analyze wldp.dll. After importing the file and running auto-analysis,



The “Symbol Table” was searched for "Sleep".

Symbol References - (Sleep: 3 References)

From Location	Label	Subroutine	Ref ...	From ...
180074a8b		thunk_FUN_1800748e0	Call	CAL...
1800749f6		thunk_FUN_1800748e0	Call	CAL...

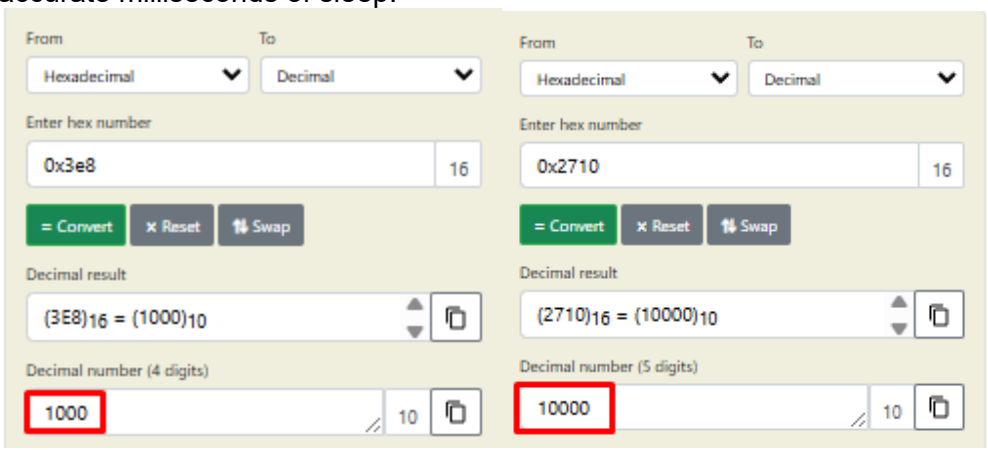
The results showed that the Sleep function from KERNEL32.DLL was called via a thunk wrapper named thunk_FUN_1800748e0.

Two call sites were identified, each loading a hardcoded value into the ECX register before calling Sleep:

```
1800749f1 b9 10 27        MOV       ECX,0x2710
          00 00
1800749f6 ff 15 1c        CALL      qword ptr [->KERNEL32.DLL::Sleep]          = 001978fe
          26 12 00
1800749fc 90              NOP
```

```
180074a86 b9 e8 03        MOV        ECX,0x3e8
          00 00
180074a8b ff 15 87        CALL       qword ptr [->KERNEL32.DLL::Sleep]          = 001978fe
          25 12 00
180074a91 90              NOP
```

Using the website https://www.rapidtables.com/convert/number/hex-to-decimal.html?x=2710
To convert Hex To Decimal, We need to convert 0x2710 And 0x3E8 to Decimal to get the accurate milliseconds of sleep.



Address	Instruction	Hex Value	Milliseconds
0x1800749F1	MOV ECX, 0x2710	0x2710	10,000 ms
0x180074A86	MOV ECX, 0x3E8	0x3E8	1,000 ms
		Total:	**11,000 ms**

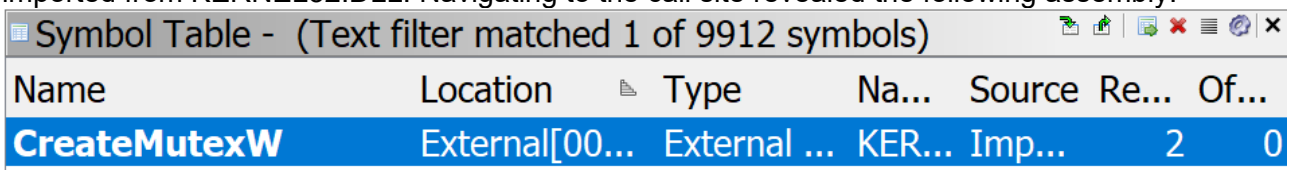
The first Sleep call occurs immediately after CreateProcessW launches browser.exe, giving the browser time to initialize. The second Sleep follows shortly after, allowing the DLL injection into the browser process to complete before the C2 shell becomes active.

Answer: `11,000 milliseconds`

Task 12 - Mutex Name

A mutex (mutual exclusion object) is a synchronization mechanism used in programming to ensure that only one instance of a process runs at a time. Malware commonly uses mutexes to prevent multiple copies of itself from running simultaneously.

In Ghidra, the Symbol Table was searched for "Mutex", which revealed a call to CreateMutexW imported from KERNEL32.DLL. Navigating to the call site revealed the following assembly:



```
180074907 48 8d 0d        LEA       RCX,[DAT_18019909f]                        = 01h
          91 47 12 00
18007490e e8 90 c6        CALL      __CheckForDebuggerJustMyCode               undefined __CheckForDebuggerJust...
          ff ff
180074913 90              NOP
180074914 4c 8d 05        LEA       R8,[u_Global\YandaExeMutex_180155d60]      = u"Global\\YandaExeMutex"
          45 14 0e 00
18007491b ba 01 00        MOV       EDX,0x1
          00 00
180074920 33 c9           XOR       ECX,ECX
180074922 ff 15 e8        CALL      qword ptr [->KERNEL32.DLL::CreateMutexW]   = 001978ee
```

The LEA (Load Effective Address) instruction loads the memory address of the string into register R8, rather than copying the string itself. This is because CreateMutexW expects a pointer to the mutex name, not the name directly.

Answer: `Global\\YandaExeMutex`

Task 13 - Full Path of the C2 Binary

The CryptnetUrlCache metadata from Task 8 revealed that two files were downloaded from the C2 server: wldp.dll and a second file named yanda.tmp. Two methods were used to find its full path on the system:

Option 1 — $MFT: The Master File Table recorded the full path when the file was written to disk.

Parent Path	File Name
ABC	ABC
.\Users\Administrator\AppData\Local\Temp	yanda.tmp

Option 2 — Ghidra: Searching the Defined Strings in wldp.dll (Search → For String → "yanda") revealed the full path hardcoded in the binary.

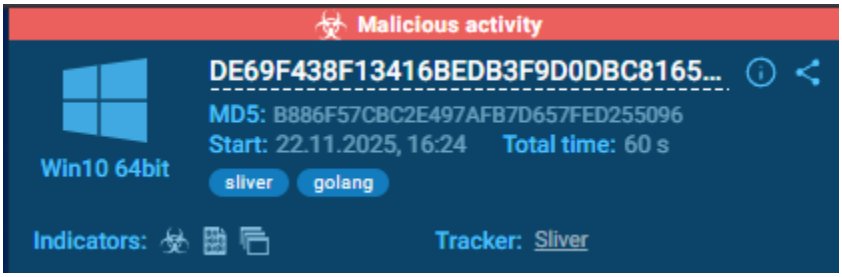
```
180074a78 48 8d 0d        LEA       RCX,[u_C:\Users\Administrator\AppData\L_180155... = u"C:\\Users\\Administrator\\Ap...
          21 14 0e 00
180074a7f ff 15 ab        CALL      qword ptr [->KERNEL32.DLL::CreateProcessW]      = 0019792e
```

= u"C:\\Users\\Administrator\\AppData\\Local\\Temp\\yanda.tmp"

Answer: `C:\Users\Administrator\AppData\Local\Temp\yanda.tmp`

Task 14 — C2 Framework

Using the SHA-256 hash of the wldp.dll file (from Task 10), a search was conducted on any.run, which returned an existing sandbox analysis. The analysis identified the implant framework as Sliver.



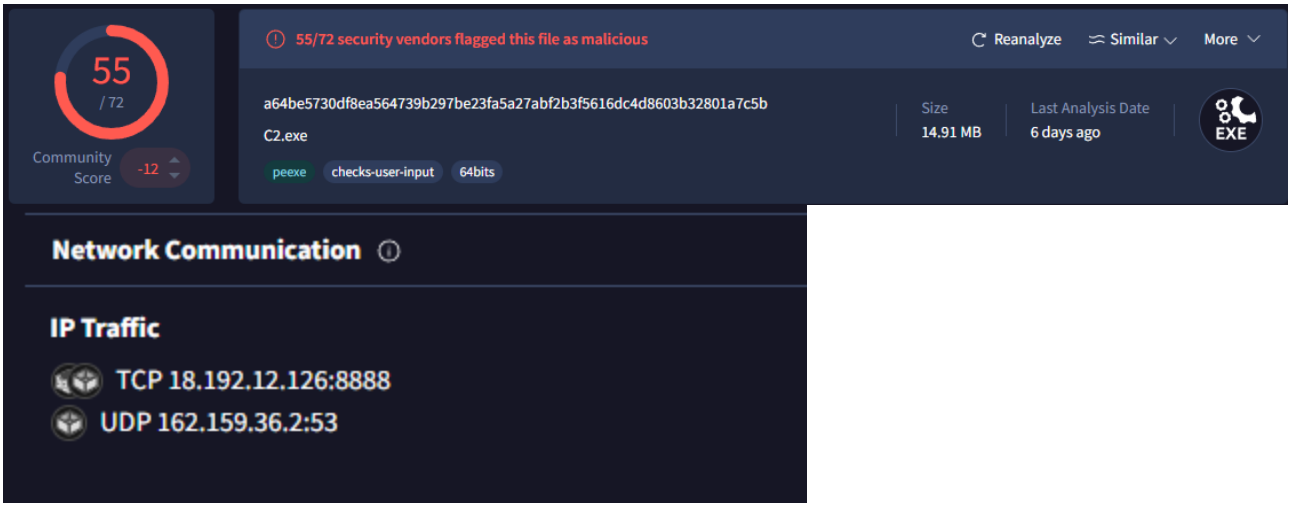
This was further corroborated by static analysis of the binary in Ghidra. The Defined Strings output revealed several Sliver-specific indicators:

- ReconnectInterval protobuf field names - characteristic of Sliver's C2 configuration structure
- IKIZRGX.Envelope - Sliver's internal protobuf message wrapper for C2 communications
- Obfuscated Go package names (e.g., Dzhiga75, qiSH7YAP) - Sliver randomizes package names at compile time
- Supported C2 protocols: mtls, https, tcp, socks5 - all consistent with Sliver's transport options

Answer: `Sliver`

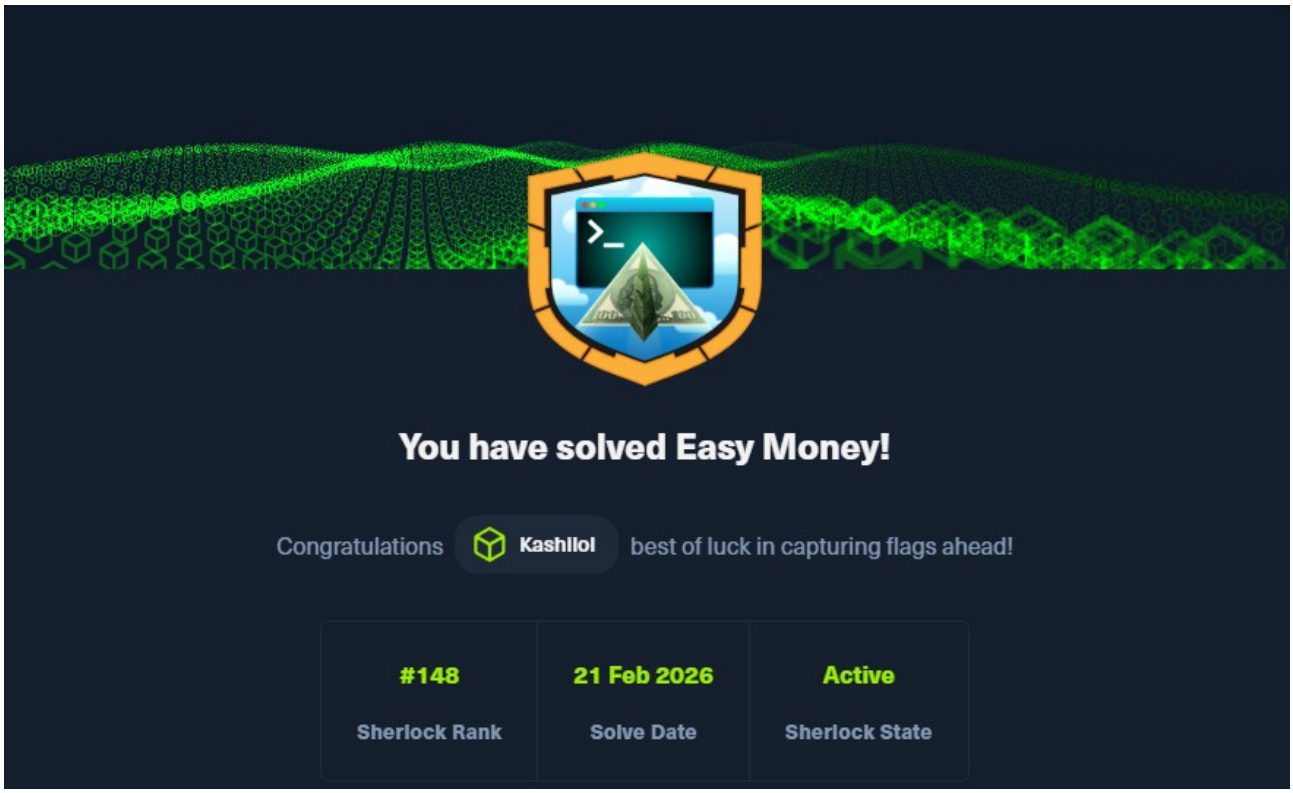
Task 15 - C2 Server IP Address and Port

The SHA-256 hash of wldp.dll was submitted to VirusTotal. Under the Behavior tab, in the Network Communication section, the C2 server IP address and port were listed as observed connections from sandbox execution.



This was also independently confirmed by the CryptnetUrlCache metadata files from Task 8, which recorded the download URLs for both payloads, showing the originating server as 18.192.12.126 on port 8000.

Answer: `18.192.12.126:8888`



End of Writeup — Sherlock: EasyMoney